

CL3410 Language Models and Agents: Final Project Report

Abhinav Borah
Roll Number: 2025204042

September 17, 2026

1 Introduction

This project builds two from-scratch, fully independent monolingual decoder-only transformer language models, roughly 25M parameters each. Model H is Hindi, a higher-resource Indian language. Model L is Assamese, a deliberately lower-resource one. No pretrained weights, no pretrained tokenizers, and no HuggingFace transformer classes are used anywhere. Embeddings, multi-head causal self-attention, and the causal mask are built from raw `nn.Linear`, `nn.Embedding`, `nn.LayerNorm`, and `nn.Dropout`. The two languages share no data, no vocabulary, and no weights at any stage, including the reasoning finetuning in Phase 3.

The project has three phases: data collection and tokenization (Phase 1), model architecture, pretraining, and evaluation (Phase 2), and reasoning finetuning with attention analysis (Phase 3). This report covers all three, plus a closing synthesis and reflections section.

2 Phase 1: Data and Tokenizer

2.1 Language choice and corpus

Hindi was chosen as the higher-resource language for its abundant public corpora and mature scraping infrastructure. Assamese was chosen from the assignment’s lower-resource list. The motivation was real digital text availability and the specific challenge of reaching the manual-collection floor for a genuinely lower-resource language.

Both corpora were built from a mix of manually collected text (OCR of textbooks, live-scraped news and dictionary sites, Wikipedia dumps processed with a project-owned script) and downloaded corpora (Sangraha and related public Indic corpora). Both clear the roughly 500M-token target and the 20 percent manual-collection floor, measured directly on the real trained tokenizer’s output.

	Hindi	Assamese
Total tokens (real, after tokenization)	850,380,387	575,890,968
Manual tokens (real)	202,330,883	137,442,545
Manual fraction	23.8%	23.9%

Cleaning applied Unicode normalization, a script-purity filter that drops any line whose alphabetic characters are not majority in-script, and exact-duplicate removal by content hash. An Assamese-specific heuristic based on the frequency of the letters ra and va catches Bengali text, since the two languages share almost the entire Unicode block. Splits are document-level, assigned deterministically by hashing each document’s content: roughly 98 percent train, 1 percent validation, 1 percent test. This follows common practice for pretraining corpora at this scale; it is a different convention from the 80/10/10 split used for smaller classification datasets.

2.2 Tokenizer

Both tokenizers are byte-level BPE trained with SentencePiece, per course guidance to use a library instead of a hand-rolled implementation. Vocab size was chosen via a fertility sweep on held-out text. Hindi settled on 8,000: the fertility curve’s largest improvement happens between 5K and 8K vocabulary size, and further increases add little. Assamese was initially estimated at 5,000 from a small proxy sample. Once the real corpus was measured, real fertility came in well ahead of the proxy estimate, so Assamese was matched to Hindi’s 8,000-token vocabulary; the proxy sample alone would have argued for keeping it smaller.

	Hindi	Assamese
Vocabulary size	8,000	8,000
Fertility (tokens per word)	1.4940	1.7780
UNK rate	0.0	0.0

Assamese’s higher fertility reflects it being the more morphologically complex, lower-resource language in this pair. This fertility gap becomes directly relevant later, when comparing perplexity across the two languages in Phase 2.

Two corpus bugs were found and fixed during Phase 1, both documented with before and after numbers. A source-list

omission had dropped three already-collected Assamese sources from the counts and splits. A cleaning-code change altered how embedded non-target-script words such as brand names are handled, which required rebuilding both corpora and retraining both tokenizers.

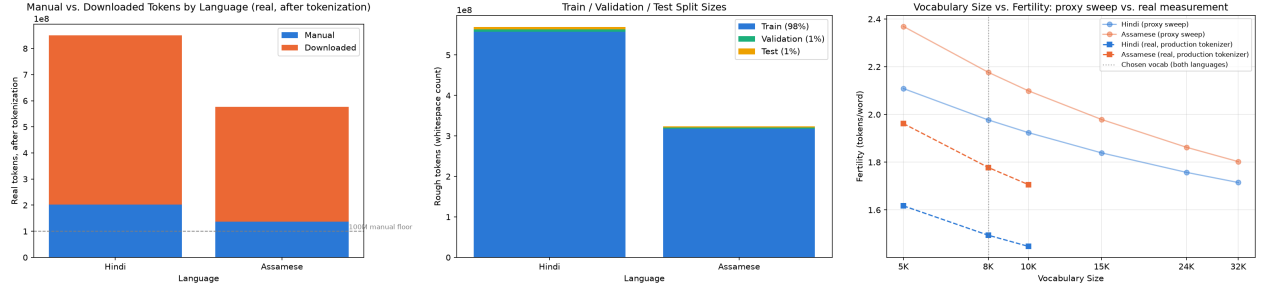


Figure 1: Manual versus downloaded tokens by language (left); train/validation/test split sizes (middle); fertility versus vocabulary size sweep (right).

3 Phase 2: Model, Pretraining, and Evaluation

3.1 Architecture

Both models share one architecture: 12 layers, d_{model} 384, 6 heads (head dimension 64), feedforward dimension 1536, context length 256, dropout 0.1, and a measured parameter count of 24,366,336 for either language.

Position is injected via RoPE (rotary position embedding), applied inside attention to the query and key vectors instead of being added at the embedding layer. This adds no parameters. Relative position falls out of the dot product algebraically. RoPE was chosen over learned absolute embeddings based on small-model literature at comparable scale. Most modern from-scratch small-LM reference implementations also default to RoPE regardless of parameter count. RoPE has no fixed embedding table to run out of rows in, unlike learned absolute embeddings, so there is no hard architectural cap on sequence length; in practice the real ceiling is the trained context length of 256 tokens, since the model never learns to interpret position values beyond what it was trained on, and that ceiling comes from training dynamics, not table size.

Multi-head causal self-attention and the causal mask are implemented directly. Queries, keys, and values come from separate learned linear projections. Attention scores are scaled by the inverse square root of head dimension. A triangular additive mask blocks attention to future positions. A dedicated test verifies the mask actually works: it checks that changing a future token never changes any earlier position’s output logits.

Each transformer block uses Pre-LN (normalization before each sublayer) instead of Post-LN. This gives simpler, more stable training dynamics at this depth with no separate warmup-schedule tuning required. Shallow-network literature suggests Post-LN might offer a small quality advantage at this depth; that tradeoff was accepted for the simplicity gain. The output projection is tied to the input token embedding, saving about 12 percent of the parameter budget for either language and putting that budget into additional transformer depth instead. Depth and width (12 layers over a shallower, wider alternative) were chosen partly because the required early, mid, and late layer attention analysis needs enough discrete layers to bucket meaningfully.

Training uses the standard causal language modeling objective, AdamW, and a warmup-then-schedule learning rate computed as a deterministic function of the saved training step. This means no separate scheduler object or state is needed to resume a run exactly, matching nanoGPT’s own approach. Every checkpoint saves model weights, optimizer state, training step, and configuration. A test proves this is resumable: it compares an uninterrupted run against a save, reload, and resume of the same run.

3.2 Training and evaluation

Both models were trained locally on an Apple M4 Pro (MPS backend) for one epoch. Training started on Colab, but a run hit an undocumented free-tier resource cap partway through Hindi training, so the project moved to local hardware.

	Hindi	Assamese
Real train tokens	834,049,861	565,255,443
True active training time	17.28 hours	11.92 hours
Test perplexity	38.46	53.67
Test bits-per-byte	0.5952	0.5990

Perplexity and bits-per-byte tell different stories about the gap between the two languages. Assamese’s test perplexity is about 40 percent higher than Hindi’s, consistent with its 32 percent smaller real training corpus. Bits-per-byte corrects for the two languages’ different tokenizer fertility by measuring against raw bytes instead of tokens, and it is nearly identical

(0.5990 versus 0.5952). Much of the raw perplexity gap is therefore a tokenization artifact; it overstates the true quality difference between the two models. Bits-per-byte is effectively tied between the two languages: on validation, Assamese is marginally lower (0.5934 versus 0.5981). BPB alone does not establish a residual gap; two other measurements do. Generation quality (BLEU, chrF, ROUGE-L, and repetition diagnostics across four decoding settings) is consistently lower for Assamese at every setting. Attention entropy is higher for Assamese at every one of the 12 layers, meaning Assamese’s attention is measurably more diffuse. Both remaining measurements agree: the residual gap traces to data scale. Architecture, parameter count, and training procedure were held identical between the two models, so the model itself is not a contributing factor.

Setting	Hindi			Assamese		
	BLEU-4	chrF	ROUGE-L	BLEU-4	chrF	ROUGE-L
Greedy	1.64	0.90	0.0154	0.27	0.83	0.0047
Temp 0.5	2.60	0.92	0.0133	0.58	0.86	0.0064
Temp 1.0	2.10	0.90	0.0110	0.46	0.81	0.0063
Temp 1.5	0.09	0.83	0.0095	0.10	0.75	0.0029

BLEU is a poor fit for open-ended generation: scores are low across every setting and both languages, including temp 1.0, where a qualitative reading of the generated text is fluent and grammatical. BLEU penalizes any deviation from the one reference continuation that happened to follow in the held-out text, and open-ended generation from a prefix has no single correct continuation. chrF, which operates on character n-grams instead of word n-grams, is more informative here and tracks coherence more sensibly, staying in a narrower 0.75 to 0.92 range and falling at temp 1.5 where the generated text degrades. Both languages show the same generation degeneration pattern documented in the neural text degeneration literature. Greedy decoding collapses into near-total repetition (88 to 90 percent repeated bigrams). Temperature 1.0 is a clear sweet spot: repetition rate under 5 percent, high lexical diversity, zero detected repetition loops. Temperature 1.5 pushes diversity further, at the cost of coherence.

Attention analysis on real held-out sentences shows neither model follows a clean early-layers-local, late-layers-long-range story. Both models show a sharp dip in entropy and attention distance specifically at layer 2. The two models never share data, vocabulary, or weights, so this shared structural feature is notable on its own.

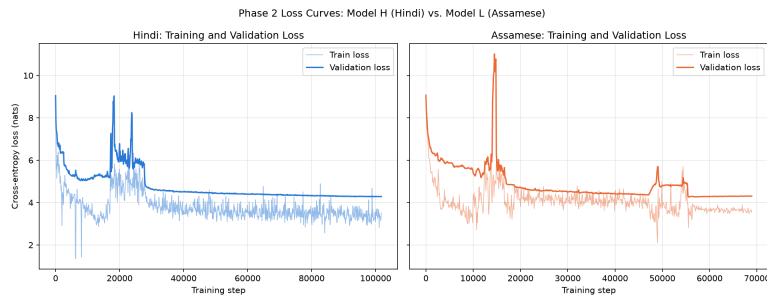


Figure 2: Training and validation loss curves, both languages. The temporary spikes in both curves line up with transitions between source domains in the unshuffled, source-ordered training split (OCR, scrape, Vikaspedia, book archive, Sangraha, in that fixed order); single-epoch training bounds the damage and both curves recover.

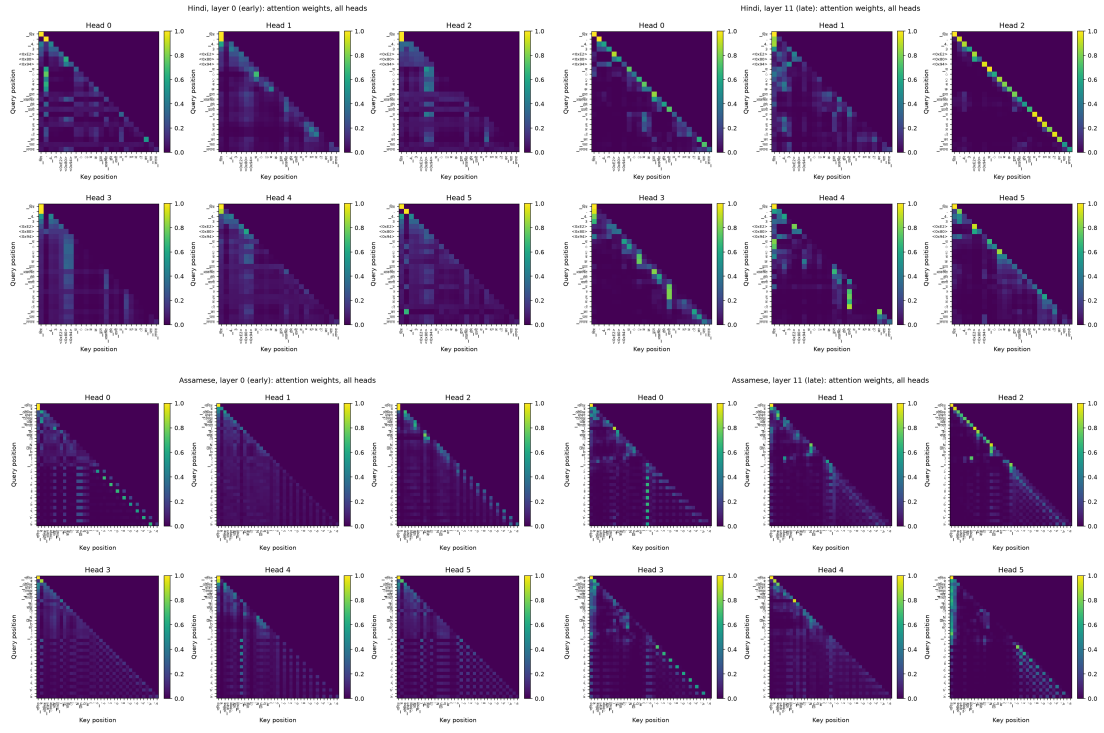


Figure 3: Attention heatmaps, early layer (left) and late layer (right), one subplot per head. Top: Hindi. Bottom: Assamese. Rows/columns are query/key position, color is attention weight.

4 Phase 3: Reasoning Finetuning and Attention

4.1 Dataset and method

Each language has its own synthetic comparative-reasoning dataset. It is generated programmatically from a deterministic function; it is not downloaded from an existing benchmark. Coverage spans four attribute domains (height, age, price, quantity), direct pairwise and multi-hop chained comparisons, and roughly 10 percent tie cases in both languages. Two leakage-avoidance axes are enforced per course guidance, each held out on its own test split. `test_unseen_entity` uses unseen entity names with trained wording. `test_unseen_wording` uses unseen phrasing with the trained entity pool. Instance-level deduplication runs within each split-generation pass but not across them: 3 of 150 Assamese `test_unseen_wording` items (2 percent) share an entity-tuple with a train or validation item, differing only in phrasing. Hindi has none.

Split	Hindi and Assamese (each)
Train	4,500
Validation	450
Test, seen wording and entities	450
Test, unseen entities	450
Test, unseen wording	150

Finetuning uses LoRA (rank 8, alpha 16, targeting the query and value projections in every attention block) instead of full finetuning. This choice rests on a direct empirical check: a 5-epoch, single-seed Hindi probe comparing full finetuning against LoRA on the same data. Full finetuning reached a near-zero reasoning-task loss, but its language-modeling perplexity on the original pretraining validation set rose by a factor of roughly 288 over those 5 epochs, against LoRA’s roughly 21 percent rise over the same 5 epochs. That is real, measured catastrophic forgetting, and it confirms the case for a rank-constrained update. The production 40-epoch LoRA runs drift further on this same diagnostic (Figure 4, right), to roughly 35 percent for Hindi and 42 percent for Assamese, still three orders of magnitude short of full finetuning’s blowup over a fraction of the training length. Rank and alpha were chosen by an empirical sweep. Both languages independently converged on rank 8 from separate real-data sweeps. Training uses 3 seeds per language, more than the course’s stated minimum of more than one seed. The epoch ceiling of 40 was kept deliberately: a diagnostic probe found that letting training run longer improves same-distribution splits while making genuine unseen-wording generalization worse.

4.2 A bug found and fixed: the termination failure

Before a fix, strict exact-match accuracy, which requires a generation to literally end with the end-of-sequence token, was 0.0000 in every cell, both languages, pretrained and finetuned. The root cause was that LoRA only targets the attention projections, so the token embedding table, including the untrained end-of-sequence row, stayed completely frozen. The finetuned model picked the correct answer with high confidence but almost never emitted the end-of-sequence token afterward. It looped a short repeating token cycle instead. The fix was implemented directly. The two affected embedding rows were made trainable through a gradient hook, since PyTorch’s `requires_grad` has no per-row granularity. They were placed in their own zero-weight-decay optimizer group so ordinary weight decay would not erode them. The fix was verified directly: on all six final checkpoints, the two affected rows now differ from the pretrained checkpoint, while 100 sampled other rows per checkpoint stay bit-identical to it.

Split	Pretrained		Finetuned (strict), Hindi / Assamese
	Hindi	Assamese	
Test, seen	0.0%	0.0%	98.4% / 65.2%
Test, unseen entities	0.0%	0.0%	96.8% / 58.2%
Test, unseen wording	0.0%	0.0%	26.2% / 43.8%

After the fix, Hindi’s strict accuracy equals its lenient accuracy exactly on every split. Every generation that gets the answer right now also terminates correctly. Assamese retains a small residual termination gap on the seen and unseen-entity splits, and shows a real, seed-consistent drop in raw content accuracy on those same splits relative to its pre-fix numbers. This is a plausible resource-tier cost of the fix, not a flaw in the fix itself: the fix routes a second, small gradient signal through the same limited LoRA-scale training budget that also has to learn the reasoning content. Assamese’s smaller, noisier pretraining signal may make it more sensitive to sharing that budget.

Hindi clearly outperforms Assamese on the seen and unseen-entity splits, consistent with the Phase 2 language-modeling gap. The direction reverses on unseen wording: Assamese scores higher than Hindi in all three seeds individually. One plausible reading is that Hindi’s higher seen-wording accuracy reflects a larger template-memorization component that, by construction, does not transfer to unseen phrasing. Assamese’s lower seen-wording accuracy would then leave comparatively more of its gain attributable to genuine reasoning transfer. This is an observation from the data, not a proven causal claim.

Seed	Seen	Hindi strict		Seen	Assamese strict	
		Unseen ent.	Unseen word.		Unseen ent.	Unseen word.
0	98.2%	96.7%	25.3%	63.6%	59.6%	44.7%
1	98.7%	97.6%	28.0%	65.6%	57.1%	45.3%
2	98.2%	96.2%	25.3%	66.4%	58.0%	41.3%

Table 1: Per-seed strict accuracy, confirming the unseen-wording reversal (Assamese above Hindi) holds in every individual seed, not the mean alone.

4.3 Attention shift after finetuning

Attention was compared on real comparative-reasoning prompts, pretrained versus finetuned, at one early layer (layer 0) and one late layer (layer 11 of 12), per the spec’s early-and-late requirement. Two examples were used: a genuine short pairwise example, and a longer chained example that requires combining two premises.

Example	Layer	Entropy		Mean distance	
		Pretrained	Finetuned	Pretrained	Finetuned
<i>Hindi</i>					
Pairwise	Early (0)	2.008	1.993	6.421	6.487
Pairwise	Late (11)	1.627	1.571	4.618	4.827
Chained	Early (0)	2.577	2.562	12.108	12.129
Chained	Late (11)	2.109	1.870	8.242	6.705
<i>Assamese</i>					
Pairwise	Early (0)	2.213	2.194	5.570	5.570
Pairwise	Late (11)	1.671	1.565	6.061	5.635
Chained	Early (0)	2.573	2.522	9.238	9.304
Chained	Late (11)	2.209	1.878	11.335	9.477

Late-layer attention entropy drops after finetuning in every language and example-type comparison, most strongly on the chained comparisons. This means finetuning teaches the model to attend more selectively at the layer closest to the output, most where the reasoning task actually requires combining multiple premises. Early-layer entropy barely moves in any of the four comparisons (at most a 2 percent relative shift), backing the claim that early-layer attention is essentially unchanged. Mean attention distance is more mixed: it shortens in three of four comparisons, but lengthens slightly for Hindi’s pairwise case. This is a single example per language per type, so the sign of that one small shift is not a robust finding on its own; it is reported here, not smoothed away.

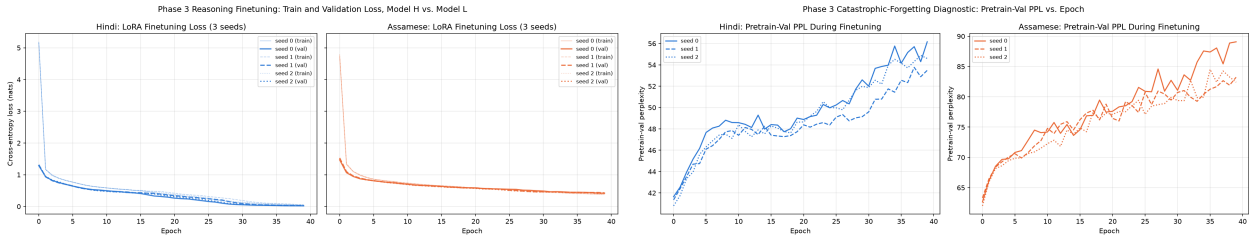


Figure 4: Left: reasoning finetuning train/validation loss, both languages, 3 seeds. Right: pretrain-validation perplexity during finetuning, the catastrophic-forgetting diagnostic.

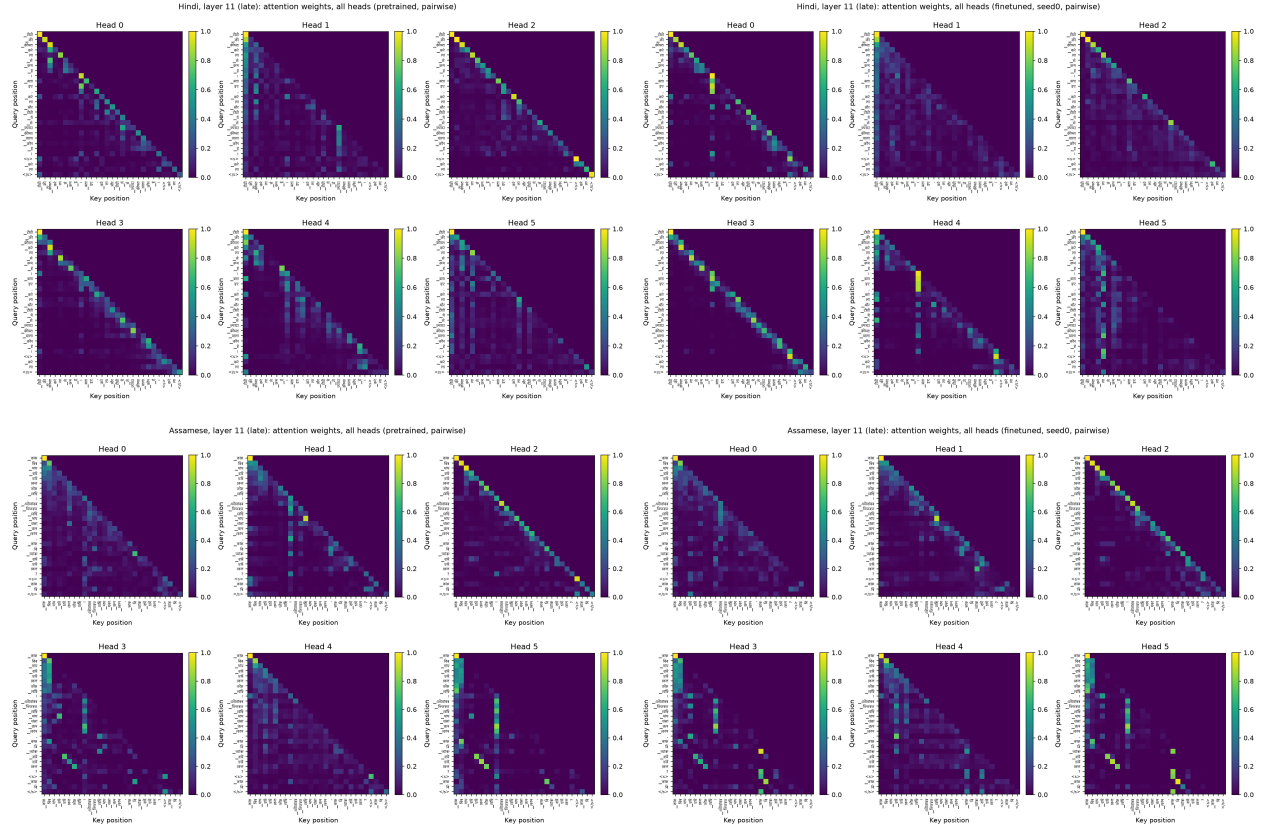


Figure 5: Late-layer attention heatmaps, pairwise example, pretrained (left) versus finetuned (right), one subplot per head. Top: Hindi. Bottom: Assamese.

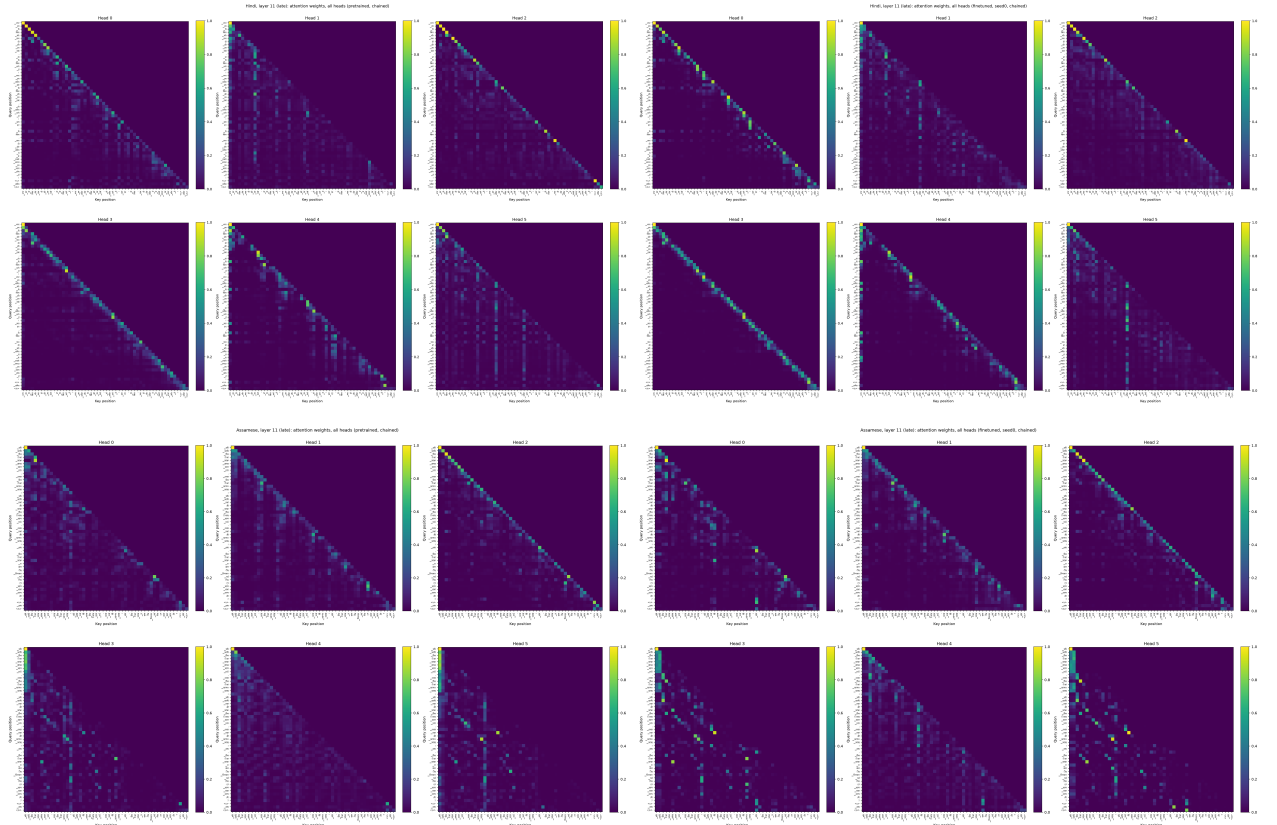


Figure 6: Late-layer attention heatmaps, chained example, pretrained (left) versus finetuned (right), one subplot per head. Top: Hindi. Bottom: Assamese.

5 Cross-Phase Synthesis

How did data scale and quality differ between Model H and Model L? Hindi’s real corpus (850,380,387 tokens) is 32 percent larger than Assamese’s (575,890,968 tokens), though both clear the 500M-token target and the 20 percent manual-collection floor by the same measurement standard. Both went through the identical cleaning pipeline, so the gap between them is a scale effect.

How do language-modeling and reasoning results compare across the two resource tiers? Hindi outperforms Assamese on test perplexity, generation quality, and reasoning accuracy on the seen and unseen-entity splits. The one reversal is unseen-wording reasoning accuracy, where Assamese scores higher in all three seeds; this is reported as a genuine exception, not forced into the same pattern.

What tokenizer or corpus factors most affected the lower-resource model? Two factors, separable via bits-per-byte. Assamese’s tokenizer has higher fertility (1.7780 versus 1.4940 tokens per word), so it needs more, smaller tokens to cover the same text; this inflates its per-token perplexity without reflecting a real quality gap, which is why bits-per-byte nearly closes the gap that raw perplexity shows. The remaining, smaller gap is corpus scale: Assamese’s 32 percent smaller real training corpus, corroborated by generation quality and attention entropy.

What evidence explains the observed differences? Three independent kinds of evidence, intrinsic language-modeling quality, generation quality, and internal attention structure, converge on corpus scale as the dominant remaining factor once tokenizer fertility is accounted for. The LoRA end-of-sequence finding separates a method-level cause from a resource-tier-dependent consequence. Before the fix, both languages failed identically: that is evidence the root cause is a property of the finetuning method, not of either language’s data. After the fix, the two languages diverge, which is itself evidence about how each language’s smaller or larger training signal responds to sharing gradient budget between an embedding fix and reasoning-content learning.

6 Reflections

When I first ran the real reasoning evaluation against the finetuned checkpoints, strict exact-match accuracy came back 0.0000 across every checkpoint, both languages, all three seeds, every split. My first instinct was to treat that as a clean, reportable finding about LoRA: it only touches the attention projections, so of course it cannot learn to terminate a generation. I nearly wrote it up that way and moved on. What stopped me was the uniformity. A 0.0000 that shows up identically across two independent languages, three different random seeds, and three splits with different held-out properties is a more specific signal than "the method has a known weakness". It reads like a bug, not a property. Chasing that suspicion instead of the tidier story is what actually led to the fix: two embedding rows never got a gradient because LoRA’s target modules do not touch the token embedding table at all, and a gradient hook plus a separate optimizer group closes that gap directly, verified checkpoint by checkpoint against the pretrained weights.

The second thing worth naming is a genuine open question I have not resolved, not a lesson already learned. After the fix, Assamese’s raw content accuracy on the seen and unseen-entity splits is measurably lower than it was before the fix, consistently across all three seeds. I have a plausible mechanism for this, that the fix adds a second small gradient signal competing with reasoning-content learning for the same limited LoRA budget, and that Assamese’s noisier pretraining signal makes it more sensitive to that competition than Hindi is. But I have not run the experiment that would actually distinguish this from other explanations, such as the fix simply interacting differently with whatever made Assamese’s finetuning noisier to begin with. If I had more time before this deadline, that is the experiment I would run next, not because the current explanation is wrong, but because I cannot currently rule out being wrong about it.

A smaller but still real lesson was about verification discipline while writing the reports themselves. More than once, a claim I had already written down turned out not to hold up when I went back and checked it against the actual source: a course clarification I assumed had reversed a cleaning rule but that I could not actually find recorded anywhere, a figure caption that called a diagnostic curve flat when the plotted data was clearly rising. None of these were large errors individually, but they were the kind that survive silently if the only check is "did I already write this down somewhere" rather than "does this match the artifact it claims to describe."

7 Limitations

The pretraining corpus is not i.i.d.-shuffled: each language’s train split is a fixed-order concatenation of source domains, which is why the loss curves in Figure 2 show temporary spikes at domain boundaries instead of a smooth curve throughout. Near-duplicate detection is not implemented for either language’s pretraining corpus, only exact-hash deduplication, budgeted at an estimated eighteen to twenty four additional machine hours and judged better spent on evaluation work the spec requires. The optional bonus ablation (retraining one language without positional embeddings and rerunning the full Phase 2 evaluation suite) was not attempted, since it requires a full second pretraining run that does not fit the project’s remaining time. A word-level filter for embedded non-target-script words was removed from the cleaning pipeline partway through Phase 1, based on an internal reading of a course clarification. A later review could not find a recorded reversal of the original guidance on the course’s own clarification page. This specific cleaning decision remains an open question the author intends to confirm directly with the course staff.